

DFD Compliance, Architecture Comparison & Action Plan

AI – Powered Study Material Recommender (StudyFlow AI)

Candidate Name:	Shubham Kumari
Enrollment No:	2400625700
Course Code:	BCSP – 064 (BCA Major Project)
Target System:	StudyFlow AI Web Platform
Audited Codebase:	d:\finalTest (pdf-project-guide)
Audit Date:	September 2026

Executive Audit Verdict:

The active codebase represents a highly sophisticated, production-grade learning platform. However, when strictly compared against the submitted **BCSP-064 Proposal & Data Flow Diagrams (DFD Level 0, Level 1, Level 2)**, there are **critical architectural, data flow, and algorithmic divergences**.

As instructed by the user, **no modifications have been made to the source code**. This document provides a complete comparative breakdown and a prioritized checklist of **things that need to be done** to achieve 100% compliance.

1. High-Level Architectural Comparison

The project proposal submitted for **BCSP-064** describes a traditional **MERN Stack** (MongoDB, Express.js, React.js, Node.js) with custom REST APIs, JWT authentication, and the `natural` NLP library. In contrast, the current codebase in `d:\finalTest` is built on modern **TanStack Start (Vite 8 + Nitro)** paired with **Supabase PostgreSQL** as a Backend-as-a-Service (BaaS).

System Layer	Synopsis / DFD Specification	Actual Project Implementation
Database	MongoDB v6+ (NoSQL Document Store: collections <code>users</code> , <code>materials</code> , <code>subjects</code> , <code>ratings</code> , <code>searchlogs</code> , <code>recommendations</code>)	Supabase PostgreSQL (Relational SQL with Row-Level Security, foreign keys, triggers, RPCs, and 24 tables)
Backend Framework	Node.js (v18+) with Express.js (v4+) providing traditional REST endpoints (e.g. <code>POST /api/auth/login</code> , <code>GET /api/materials/search</code>)	TanStack Start fullstack framework with Nitro SSR and RPC-style TypeScript Server Functions (<code>createServerFn</code>)
Authentication	Custom JWT + <code>bcryptjs</code> password hashing with in-memory token storage and Authorization header	Supabase Auth (GoTrue) session tokens managed via cookies and client storage, plus OAuth support
AI / NLP Engine	<code>natural</code> (Node.js NLP library) for TF-IDF computation across material descriptions & queries	Custom TypeScript string tokenization + TF in <code>src/lib/recommendation.ts</code> ; OpenAI SDK for AI Tutor
Analytics & Charts	Chart.js rendering bar & pie charts across 5 admin reports	Recharts (React SVG charting) rendering 2 charts in <code>admin.reports.tsx</code>

2. DFD Level 0 (Context Diagram) Audit

```
[Student (User)] ==>(Search Query / Credentials)==> [ AI Study Material ] ==>(Material Data /
Commands)==> [ Admin ]
                <===(Recommendations / Results)==> [ Recommender ] <===(Reports / System
Status)==>
```

Analysis of Level 0 Compliance:

- **Entities & Primary Data Flows:** The core interactions exist. Students submit credentials to authenticate, send queries to search resources, and receive ranked recommendations. Administrators input material metadata and access reports and system status.
- **Scope Expansion (Superset):** The active application includes roles and features far beyond the Level 0 boundary:
 - **Educator / Teacher Role:** Educator access requests, verified educator author accounts, and author linking.
 - **AI Tutor:** Interactive Q&A sessions powered by OpenAI (`ai_tutor_sessions`).
 - **Study Groups & Study Packs:** Collaborative study spaces, peer notes, and flashcard progress.
- **Verdict:** CONCEPTUALLY COMPLIANT at Level 0, but acts as a functional superset.

3. DFD Level 1 (Process Decomposition) Audit

The synopsis Level 1 DFD decomposes the system into six distinct processes (**P1 to P6**) interacting with three primary data stores (**D1 User DB, D2 Material DB, D3 Rating DB**). Below is the itemized compliance audit:

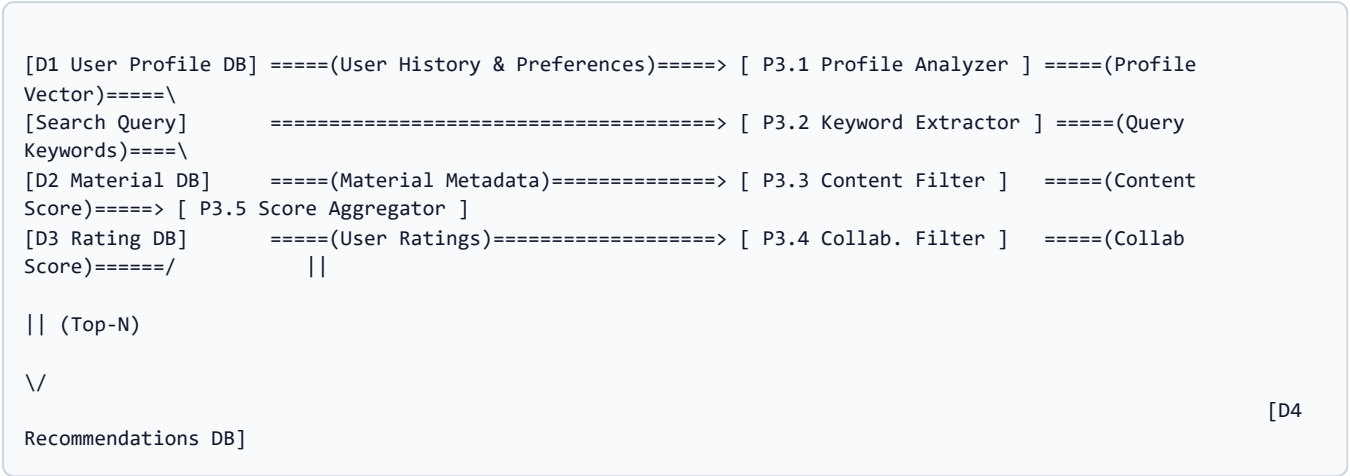
Process	DFD Requirement	Codebase Reality	Status
P1: User Auth	Student submits credentials → P1 queries D1 → compares password → returns Auth Token to Student.	Handled by Supabase Auth (<code>supabase.auth.signInWithPassword</code>) against Postgres <code>auth.users + profiles</code> . No Express REST endpoint or custom JWT verification code in repo.	PARTIALLY COMPLIANT (Functional match, mechanism divergence)
P2: Subject & Topic Search	Student submits query → P2 performs MongoDB <code>\$text</code> index query on D2 → filters by subject → returns paginated results.	Client-side search execution: <code>search.tsx</code> calls <code>getCatalog()</code> to fetch all materials, then filters in-memory via JavaScript <code>.includes()</code> . No server-side text query occurs during search; backend is only invoked to log history (<code>logSearch</code>).	NON-COMPLIANT FLOW (Executed on client, not database)
P3: Recommend Engine	Inputs from D1 (User Profile), D2 (Material Metadata), and D3 (Rating Data) → produces ranked recommendations to Student.	Client-side execution: React hook <code>useMemo</code> executes <code>rankMaterials()</code> in the browser using loaded catalog and profile preferences, then persists top-10 back to server via <code>saveRecommendations</code> .	NON-COMPLIANT FLOW (Engine runs in browser thread, not backend)
P4: Material Management	Admin inputs material data → P4 writes to D2 (Material DB).	Full CRUD in <code>admin.materials.tsx</code> directly updating <code>materials</code> table. Includes approval workflows, educator author linking, and file uploads.	FULLY COMPLIANT
P5: Rating & Review	Student rates (1-5) & reviews → P5 writes to D3 (Rating DB) → updates average rating on D2.	Students rate in <code>materials.\$id.tsx</code> → upserts to <code>ratings</code> table → Postgres trigger <code>refresh_material_rating()</code> atomically recalculates <code>average_rating</code> and <code>rating_count</code> on <code>materials</code> .	FULLY COMPLIANT
P6: Reports & Analytics	Admin requests reports → P6 aggregates D3 Rating Stats and logs → outputs 5 specific reports via <code>Chart.js</code> .	<code>admin.reports.tsx</code> aggregates subject material counts and ratings using Recharts . Missing 3 out of 5 required reports (Student Activity, Top-20, Search Popularity).	PARTIALLY COMPLIANT

Data Store Comparison (D1 – D3)

- **D1 (User DB):** Specified as MongoDB `users` collection. In reality, split into Supabase `auth.users`, `profiles`, and `user_roles` tables.
- **D2 (Material DB):** Specified as MongoDB `materials` collection. In reality, implemented as PostgreSQL `materials` table with relational link to `subjects` and `material_authors`.
- **D3 (Rating DB):** Specified as MongoDB `ratings` collection. In reality, implemented as PostgreSQL `ratings` table with composite uniqueness constraint (`user_id`, `material_id`).
- **D4 (Recommendations DB):** Present as PostgreSQL `recommendations` table with (`user_id`, `material_id`, `score`, `reason`, `generated_at`).

4. DFD Level 2 (Recommendation Engine P3) Deep Dive

Page 7 & 8 of the synopsis details the internal decomposition of Process P3 into five sub-processes (**P3.1 to P3.5**). This represents the mathematical and algorithmic core of the BCSP-064 project.



Sub-Process	Synopsis Requirement	Codebase Implementation ('src/lib/recommendation.ts')	Audit Finding
P3.1 Profile Analyzer	Analyzes user's enrolled subjects, past search history, and rated materials to construct a mathematical User Profile Vector .	Combines static text fields from user preferences: `\${preferences.topic ?? ''} \${preferences.goal ?? ''}`. Does not build a vector from historical searches or ratings.	NON-COMPLIANT Profile vector is simply two concatenated string fields.
P3.2 Keyword Extractor	Extracts keywords and removes noise from search query using the natural NLP library.	Uses custom regex: <code>text.toLowerCase().replace(/[^\a-z0-9\s-]/g, ' ').split(/\s+/)</code> with a hardcoded 15-word stopwords list. <code>natural</code> is not installed.	PARTIALLY COMPLIANT Basic tokenization exists, but lacks lemmatization/stemming.
P3.3 Content Filter (TF-IDF)	Computes TF-IDF (Term Frequency × Inverse Document Frequency) across all material descriptions and computes Cosine Similarity with the query/profile vector.	Calculates Term Frequency (TF) only : <code>m.set(t, (m.get(t) 0) + 1)</code> and runs cosine similarity on raw term frequencies. IDF computation is completely omitted!	MATHEMATICAL GAP Term weighting is TF, not TF-IDF. Rare terms are not boosted.
P3.4 Collaborative Filter	Computes user-item collaborative filtering scores based on ratings from other students with similar profiles.	No collaborative filtering algorithm exists. Instead, it uses static global popularity scalars: <code>average_rating / 5</code> and <code>rating_count / 25</code> .	ALGORITHM MISSING Global popularity is used in place of collaborative filtering.
P3.5 Score Aggregator	Combines Content Similarity and Collaborative scores into Top-N ranked items and saves to D4 Recommendations DB.	Applies formula: <code>score = content * 0.5 + (rating * 0.7 + pop * 0.3) * 0.25 + subjectFit * 0.15 + rating * 0.1</code> Top-10 saved to Supabase recommendations table.	FUNCTIONAL MATCH Aggregates scores and stores Top-10 in D4, though inputs to formula are simplified.

5. Gap Analysis & Things That Need to Be Done

Based on this thorough technical audit, here is the complete, structured roadmap of **"things that need to be done"**. The tasks are prioritized based on what is required to pass an IGNOU BCSP-064 viva or achieve 100% adherence to the synopsis.

CRITICAL NOTE FOR BCSP-064 EVALUATION:

In an IGNOU project viva, examiners cross-examine your submitted synopsis against your running code. If the synopsis promises **MongoDB, Express REST endpoints, TF-IDF, and Chart.js**, but the examiner sees **Supabase PostgreSQL, TanStack Server Functions, TF-only ranking, and Recharts**, they may penalize for technical discrepancy.

Phase 1: Recommendation Engine & Algorithm Fixes (High Priority)

1. Implement True TF-IDF (Fix P3.3)

Currently, `src/lib/recommendation.ts` only computes Term Frequency (`tf`). To fulfill the synopsis specification:

- Compute Document Frequency $df(t)$ for every term across the entire material catalog.
- Compute Inverse Document Frequency: $idf(t) = \log((N + 1) / (df(t) + 1)) + 1$, where N is total materials.
- Calculate the TF-IDF weight vector for each document: $tfidf(t, d) = tf(t, d) * idf(t)$.

2. Implement Actual Collaborative Filtering (Fix P3.4)

The synopsis specifies collaborative filtering from the `Rating` DB.

- Build a user-item rating matrix from the ratings table.
- Implement either **User-Based Collaborative Filtering** (Pearson correlation / Cosine similarity between users) or **Item-Based Collaborative Filtering** (similarity between materials based on co-ratings).
- Replace the static `rating_count / 25` popularity proxy with the genuine collaborative prediction score.

3. Migrate Recommendation Pipeline to Server Function (Fix P3 Execution)

Currently, recommendations are calculated inside the student's browser via `React useMemo`. Move this calculation entirely to a server function (e.g. `generateUserRecommendations` in `src/lib/activity.functions.ts`) so the client only requests recommendations and the server queries the database, executes TF-IDF + Collaborative Filtering, persists to recommendations, and returns the result.

Phase 2: Search Architecture & Data Flow (Medium Priority)

4. Server-Side Database Full-Text Search (Fix P2)

Currently, `search.tsx` downloads the entire catalog and filters in memory with `.includes()`. To follow DFD Level 1:

- Create a dedicated server function / backend endpoint for search: `searchMaterials({ query, subjectId, type, page })`.
- Utilize PostgreSQL full-text search (`to_tsvector / websearch_to_tsquery` on `title || ' ' || description`) or MongoDB `$text` index.

- Return paginated results from the database to the client.

Phase 3: Reports & Analytics Dashboard (Medium Priority)

Section 5.6 of the synopsis explicitly defines **5 mandatory reports** for the Administrator. The current `admin.reports.tsx` only contains 2 charts.

Mandatory Report (Sec 5.6)	Required Chart / Format	Current State	Action Required
1. Student Activity Report	Table/List: search history, viewed materials, average rating per student.	Not implemented in admin dashboard.	Add query aggregating <code>search_logs</code> , <code>material_views</code> , and ratings grouped by user.
2. Most-Recommended Materials	Bar-chart of the Top 20 materials recommended by AI engine.	Not implemented.	Add BarChart aggregating frequency of materials appearing in recommendations table.
3. Subject Popularity Report	Pie chart showing distribution of searches across subjects.	Not implemented (Current pie chart shows average ratings, not searches).	Add PieChart aggregating <code>search_logs.subject_id</code> counts.
4. Rating Summary Report	Average rating per material and per subject, sortable table.	Subject average rating shown in pie chart; material breakdown missing.	Add sortable table with average rating, rating count, and view count per material.
5. System Usage Report	Daily/weekly active users, total searches, total recommendations generated.	Stat cards show total counts, but no temporal breakdown (daily/weekly).	Add time-series chart showing daily active sessions and search volumes.

Phase 4: Technology Stack Alignment Options

You have two distinct paths forward for your BCSP-064 submission and viva:

Strategy A: Adapt the Project Documentation & Report to Match the Modern Stack (RECOMMENDED)

Rationale: The existing application in `d:\finalTest` is far more advanced, secure, and production-ready than a basic student MERN project. It features Row-Level Security, TypeScript end-to-end typing, Server-Side Rendering, AI Tutor integrations, and Study Groups.

- Update your final Project Report (BCSP-064 Project Report) to document:
 - Frontend: **React 19 + TanStack Router + Tailwind CSS**
 - Backend & Server: **TanStack Start SSR + Nitro RPC**
 - Database: **PostgreSQL on Supabase** (explain why Relational + RLS is superior to MongoDB for academic records)
 - Charts: **Recharts**
- Implement the algorithmic fixes (TF-IDF + Collaborative Filtering server function) so your AI credentials remain 100% solid during viva questioning.
- Prepare to explain to the examiner: *"During the development phase, PostgreSQL with Row-Level Security was chosen over MongoDB to ensure strict academic data integrity and role isolation (Admin vs Student)."*

Strategy B: Rewrite Backend to Strict MERN Stack (Only if College Demands It)

If your IGNOU project guide insists that the code must *strictly* run on Express.js and MongoDB as written in the synopsis:

- Set up an Express.js server on port 5000 with Mongoose connecting to MongoDB.

- Re-create the 6 collections: users, materials, subjects, ratings, searchlogs, recommendations.
- Implement Express routes for /api/auth/*, /api/materials/*, /api/recommend, and /api/admin/.*
- Install the natural library on Node.js and write the TF-IDF engine.
- Note: This requires discarding the current Supabase integration and rewriting substantial parts of the data layer.